



CARDMARKET CARD DETAILS SCRAPER - COMPLETE DOCUMENTATION

TABLE OF CONTENTS

1. [Overview & Purpose](#)
2. [Architecture & Design Philosophy](#)
3. [Core Components](#)
4. [Data Flow & Processing Pipeline](#)
5. [Configuration Parameters](#)
6. [Input Specification](#)
7. [Output Specification](#)
8. [Price Extraction Logic](#)
9. [N/A Detection & Priority System](#)
10. [Session Management & Rate Limiting](#)
11. [Chunk-Based Processing Architecture](#)
12. [Graceful Shutdown Mechanism](#)
13. [Error Handling & Retry Logic](#)
14. [Phase 1: Fast Parallel Extraction](#)
15. [Phase 2: Careful Recovery](#)
16. [Checkpoint & Resume System](#)
17. [Performance Characteristics](#)
18. [Progress Display System](#)
19. [Thread Safety & Concurrency](#)
20. [Troubleshooting & Common Issues](#)
21. [Best Practices & Usage Guidelines](#)

1. OVERVIEW & PURPOSE

What This Script Does

The Cardmarket Card Details Scraper is a production-grade Python web scraper designed to extract detailed pricing information for Yu-Gi-Oh! trading cards from [Cardmarket.com](https://cardmarket.com/en/YuGiOh) (cardmarket.com/en/YuGiOh). It processes card data from a JSON input file, scrapes individual card pages for comprehensive price information, and outputs structured data with extensive error handling and recovery mechanisms.

Primary Objectives

1. **Extract 5 price points per card:** Low price, trend price, and 30-day, 7-day, 1-day averages
2. **Handle large datasets:** Efficiently process tens of thousands of cards (tested with 64,000+ cards)
3. **Maximize success rate:** Two-phase architecture ensures minimal data loss
4. **Maintain data integrity:** Comprehensive validation, deduplication, and error categorization
5. **Enable graceful interruption:** User can stop execution cleanly with Ctrl+C
6. **Support resumption:** Checkpoint system allows continuing from interruption point

Key Features

- **Chunk-based processing:** Processes cards in batches of 300 for optimal balance between speed and shutdown responsiveness
- **Two-phase extraction:** Fast parallel run followed by careful recovery of failures
- **N/A price detection:** Intelligently identifies cards with no market data (priority check)
- **Session pooling:** Reuses HTTP sessions to reduce overhead and avoid rate limiting
- **CloudScraper integration:** Bypasses Cloudflare protection automatically
- **Real-time progress display:** Shows Success/Failed/N/A breakdown every 100 cards
- **Thread-safe operations:** Concurrent processing with proper synchronization
- **Comprehensive error categorization:** Separates unreachable, missing data, N/A, and input errors
- **Resume capability:** Can restart from exact stopping point after interruption
- **Zero data loss guarantee:** All completed work saved before graceful exit

2. ARCHITECTURE & DESIGN PHILOSOPHY

Two-Phase Extraction Strategy

The scraper uses a **two-phase approach** to maximize data extraction while maintaining reasonable execution times:

Phase 1: Fast Parallel Extraction

- **Goal:** Extract as many cards as possible quickly
- **Method:** 20 concurrent workers, aggressive speed (8 requests/second)
- **Retry strategy:** Limited retries (3 attempts, 3-5 second delays)
- **Expected outcome:** 90-95% success rate
- **Speed:** ~28,800 cards/hour theoretical maximum

Phase 2: Careful Recovery

- **Goal:** Recover cards that failed due to temporary issues
- **Method:** Sequential processing, conservative approach
- **Retry strategy:** Extensive retries (5 attempts, escalating 15-90 second delays)
- **Target:** Cards marked as "site unreachable" (HTTP errors, timeouts)
- **Exclusions:** N/A prices, missing data, input validation errors (permanent failures)

Chunk-Based Processing Design

Why Chunks?

The script processes cards in **chunks of 300** rather than submitting all futures at once. This architectural decision solves a critical problem:

Problem: When using Python's `ThreadPoolExecutor` with `as_completed()`, if you submit all 64,000 futures upfront:

- User presses Ctrl+C
- `KeyboardInterrupt` is raised
- But all 64,000 futures are already queued
- Workers continue processing the entire queue (takes 2+ minutes to drain)
- User cannot stop quickly

Solution: Process in manageable chunks

- Submit only 300 futures per chunk
- Process all 300 completely
- Check for interruption **between chunks** (safe point)
- If interrupted, exit after current chunk completes
- Maximum shutdown time: ~60 seconds (time to process 300 cards)

Chunk Size Rationale:

- **Too small (50-100 cards):** Excessive overhead from context switching, slower overall speed
- **Too large (1000+ cards):** Unacceptable shutdown time (several minutes)
- **Optimal (300 cards):** At 18,000 cards/hour processing speed = 60 seconds worst case shutdown
- **Compromise:** Minimal performance impact, acceptable user experience

Session Management Philosophy

Session Pooling Strategy:

Instead of creating a new HTTP session for every request (expensive and slow), the scraper:

1. **Creates one session per worker** (20 total sessions)
2. **Reuses each session 15 times** before recreation
3. **Rotates User-Agent strings** across sessions to simulate different browsers
4. **Tracks HTTP 403 errors** per session and refreshes immediately
5. **Thread-safe access** to session pool via locks

Benefits:

- Reduces connection overhead by 93% (15 requests per session vs 1)
- Avoids triggering rate limits (appears as different users)
- Automatic recovery from Cloudflare blocks (403 detection)
- Significant performance improvement (20-30% faster)

Error Handling Philosophy

Progressive Degradation:

The scraper categorizes failures into permanent and temporary:

Permanent failures (no retry):

- N/A prices - Market data simply doesn't exist
- Missing price fields - Page structure incompatible
- Invalid input data - Source data quality issue

Temporary failures (retry in Phase 2):

- HTTP 403/404/429/500/503 - Server-side issues
- Network timeouts - Connectivity problems
- Connection errors - Infrastructure issues

Benefit: Doesn't waste time retrying permanent failures, focuses recovery effort efficiently

3. CORE COMPONENTS

Class: RateLimiter

Purpose: Enforces global request rate limit across all worker threads

Mechanism:

```
min_interval = 1.0 / requests_per_second # 8 req/sec = 0.125s interval
```

Algorithm:

1. Before each request, acquire lock (thread-safe)
2. Calculate time since last request
3. If less than minimum interval, sleep remaining time
4. Add random jitter (0-0.1 seconds) to avoid thundering herd
5. Update last request timestamp
6. Release lock

Thread Safety: Uses `threading.Lock()` to coordinate across all 20 workers

Why Global? [Cardmarket.com](https://cardmarket.com) enforces rate limits per IP, not per session. All workers must coordinate.

Class: SessionPool

Purpose: Manages pool of CloudScraper sessions with intelligent refresh logic

Data Structures:

- `sessions` (dict): {worker_id: cloudscraper_instance}
- `session_uses` (dict): {worker_id: usage_count}
- `last_403_time` (dict): {worker_id: timestamp_of_last_403}

Session Lifecycle:

1. Acquisition:

```
scraper, is_new = session_pool.get_session(worker_id)
```

- Check if worker has existing session with < 15 uses
- If yes, increment counter and return existing
- If no, create new session with unique User-Agent

2. Usage Tracking:

- Each `get_session()` call increments usage counter

- After 15 uses, session is discarded and recreated
- Prevents session staleness

3. 403 Handling:

```
session_pool.mark_403(worker_id)
session_pool.refresh_session(worker_id)
```

- If worker receives HTTP 403, mark timestamp
- Force immediate session recreation
- Wait 30 seconds before allowing session reuse (Cloudflare cooldown)

User-Agent Rotation:

- 10 different User-Agent strings (Chrome, Firefox, Edge, Safari)
- Assigned via `worker_id % 10`
- Simulates traffic from different browsers
- Reduces fingerprinting

Thread Safety: All methods use `threading.Lock()` for exclusive access

Function: `parse_price()`

Purpose: Universal price parser that handles any currency format

Return Value: (`price_value: float|None`, `is_na: bool`)

Parsing Strategy:

Step 1: N/A Detection (Priority check)

```
if text in ['n/a', 'N/A', 'n.a.', 'na', '---', '----', '-', '-']:
    return (None, True)
```

Step 2: Extract numeric characters

```
cleaned = ''.join(c for c in text if c.isdigit() or c in '.,-')
```

Keeps only: digits, dots, commas, hyphens

Step 3: Decimal separator detection

Handles both European and US formats:

- European: 1.234,56 € → dot is thousands, comma is decimal
- US/UK: 1,234.56 → comma is thousands, dot is decimal

Algorithm:

```

if comma_count == 0 and dot_count == 1:
    # Simple case: "0.50"
    return float(cleaned)

elif comma_count == 1 and dot_count == 0:
    # European: "0,50"
    return float(cleaned.replace(',', '.'))

elif comma_count > 0 and dot_count > 0:
    # Mixed: Check which comes last
    if comma appears after dot:
        # European: "1.234,56"
        remove dots, replace comma with dot
    else:
        # US: "1,234.56"
        remove commas

```

Examples:

```

parse_price("0,50 €")      → (0.50, False)
parse_price("$1,234.56")   → (1234.56, False)
parse_price("£0.02")       → (0.02, False)
parse_price("1.234,56 €")  → (1234.56, False)
parse_price("N/A")         → (None, True)
parse_price("--")          → (None, True)
parse_price("invalid")     → (None, False)

```

Error Handling: Returns (None, False) on parsing failure (not N/A, just unparseable)

Function: extract_price_data()

Purpose: Extracts all 5 price fields from HTML with N/A priority checking

Return Value: (prices: dict|None, has_na: bool)

HTML Structure Expected:

```

<dt>From</dt>
<dd>2,50 €</dd>

<dt>Price Trend</dt>
<dd>3,00 €</dd>

<dt>30-days average price</dt>
<dd>N/A</dd>

<dt>7-days average price</dt>
<dd>2,75 €</dd>

<dt>1-day average price</dt>
<dd>2,80 €</dd>

```

Extraction Algorithm:

Step 1: Find all <dt> elements

```
dts = soup.find_all('dt')
```

Step 2: Match labels to price fields

```
label_map = {  
    'from': 'low_price',  
    'price trend': 'trend_price',  
    '30-day': 'avg_30_price',  
    '7-day': 'avg_7_price',  
    '1-day': 'avg_1_price'  
}
```

Step 3: For each dt, find corresponding dd

```
for dt in dts:  
    dt_text = dt.get_text(strip=True).lower()  
  
    if 'from' in dt_text:  
        dd = dt.find_next_sibling('dd')  
        price_text = dd.get_text(strip=True)  
        price_value, is_na = parse_price(price_text)
```

Step 4: Track counts

- `found_count`: How many price fields found in HTML
- `na_count`: How many are N/A
- `valid_count`: How many are valid numeric prices

Priority Return Logic:

```
# PRIORITY 1: Check for ANY N/A (even if fields missing)  
if na_count > 0:  
    return (None, True)  # Has N/A flag  
  
# PRIORITY 2: Check if all 5 fields found  
if found_count < 5:  
    return (None, False)  # Missing fields  
  
# PRIORITY 3: Check if all found prices valid  
if valid_count < 5:  
    return (None, False)  # Invalid/unparseable  
  
# SUCCESS: All 5 prices valid  
return (prices_dict, False)
```

Critical Design: N/A check happens BEFORE missing field check. Example:

Card has:

- From: 2.50€ (valid)
- Trend: Missing entirely
- 30d: N/A
- 7d: N/A
- 1d: N/A

Result: (None, True) → Classified as "N/A" (not "missing data")

Reason: The presence of ANY N/A indicates no market data exists

4. DATA FLOW & PROCESSING PIPELINE

Complete Request Flow

```
1. Main() loads input file
  ↓
2. Check for duplicates
  ↓
3. Load existing progress (checkpoint, successful cards, rejections)
  ↓
4. Filter out already processed cards (duplicate check)
  ↓
5. Phase 1: Process remaining cards in chunks
  ↓
6. For each chunk (300 cards):
   | Submit all 300 futures to ThreadPoolExecutor
   | Each worker processes card:
   |   | Validate input
   |   | Fetch HTML (rate limited, with retries)
   |   | Extract 5 prices
   |   | Check N/A (priority)
   |   | Check missing fields
   |   | Return success or rejection
   | Collect results from as_completed()
   | Display progress every 100 cards
   | Save checkpoint every 100 cards
   | Save chunk checkpoint after completion
  ↓
7. Check for KeyboardInterrupt between chunks
  ↓
8. If interrupted: Save final state, exit
  ↓
9. Phase 2: Recover "site unreachable" rejections
  ↓
10. Save final outputs
    ↓
11. Display summary statistics
```

Single Card Processing Flow

```
Card Input
↓
validate_input_card()
├─ Check required fields exist
├─ Check expansion_code not empty
├─ Check card_number not empty
└─ Check URL format
↓
[If invalid] → Return rejection "missing input data"
↓
fetch_card_details()
├─ Rate limiter wait
├─ HTTP GET request (timeout 15s)
├─ Handle status codes:
│   ├── 200: Success
│   ├── 403: Mark session, retry with delay
│   ├── 404: Reject immediately
│   ├── 429: Wait and retry
│   ├── 500/503: Short wait and retry
│   └─ Other: Reject
└─ Return (html, error)
↓
[If fetch failed] → Return rejection "site unreachable"
↓
extract_price_data(html)
├─ Parse HTML with BeautifulSoup
├─ Find all <dt> elements
├─ Match labels to 5 price fields
├─ Parse each price with parse_price()
├─ Track: found_count, na_count, valid_count
└─ Return (prices, has_na)
↓
[Check priority 1] → If has_na == True
    ↓
    Return rejection "N/A"
↓
[Check priority 2] → If prices == None
    ↓
    Return rejection "missing data"
↓
Build output structure
├─ card_data (id, name, rarity, number, set_number)
├─ expansion_data (id, name, code)
└─ price_data (url, 5 prices, date, currency)
↓
Return success with output
```

5. CONFIGURATION PARAMETERS

Phase 1 Configuration

```
PHASE1_MAX_WORKERS = 20
```

Purpose: Number of concurrent threads

Impact:

- Higher = Faster processing but more aggressive on server
- Lower = Slower but more polite, less likely to trigger rate limits

Recommended range: 12-25 workers

Current setting rationale: 20 provides optimal balance (tested empirically)

```
PHASE1_REQUESTS_PER_SECOND = 8
```

Purpose: Global rate limit across all workers

Impact:

- Theoretical max: 20 workers × 8 req/sec = 160 req/sec
 - Actual: Rate limiter enforces 8 req/sec total (bottleneck)
 - Maximum cards/hour: 8 × 3600 = 28,800
- Why not higher?** [Cardmarket.com](https://cardmarket.com) rate limits. Higher values trigger HTTP 429/403.

```
PHASE1_MAX_RETRIES = 3
```

Purpose: Retry attempts per card in Phase 1

Impact:

- More retries = Higher success rate but slower
 - Fewer retries = Faster but more failures (recovered in Phase 2)
- Current setting:** Balanced for speed vs quality

```
PHASE1_RETRY_DELAY = (3, 5)
```

Purpose: Random delay range between retries (seconds)

Impact:

- Longer delays = Better recovery from temporary issues
 - Shorter delays = Faster but may hit same error
- Why random?** Avoids synchronized retry storms from multiple workers

```
PHASE1_CHUNK_SIZE = 300
```

Purpose: Cards per processing chunk

Impact:

- Larger chunks = Faster overall (less overhead) but longer shutdown time
- Smaller chunks = Faster shutdown but more overhead

Calculation: 300 cards at ~18,000/hour = ~60 seconds per chunk

Shutdown time: Maximum 60 seconds (if interrupted at chunk start)

Phase 2 Configuration

```
PHASE2_MAX_WORKERS = 1
```

Purpose: Sequential processing in recovery phase

Why 1? Careful, conservative approach. Avoids triggering blocks.

```
PHASE2_REQUESTS_PER_SECOND = 1
```

Purpose: Very slow rate limit

Why so slow? Cards in Phase 2 already failed once. Polite retry approach.

```
PHASE2_MAX_RETRIES = 5
```

Purpose: More retry attempts than Phase 1

Why more? These cards are problematic, need extra attempts

```
PHASE2_RETRY_DELAYS = [15, 30, 45, 60, 90]
```

Purpose: Escalating delays between retries

Strategy:

- Attempt 1: Wait 15 seconds
- Attempt 2: Wait 30 seconds
- Attempt 3: Wait 45 seconds
- Attempt 4: Wait 60 seconds
- Attempt 5: Wait 90 seconds

Total time per card: Up to 240 seconds (~4 minutes) worst case

Why escalating? Server issues often temporary. Longer waits improve recovery.

Session Configuration

```
SESSION_REUSE_COUNT = 15
```

Purpose: Requests per session before recreation

Impact:

- Higher = More efficient (less overhead) but risk of session staleness
- Lower = Fresh sessions but more overhead

Why 15? Empirical testing showed optimal balance

```
RANDOM_JITTER = 0.1
```

Purpose: Random delay added to rate limiter (seconds)

Why? Prevents synchronized requests across workers (thundering herd)

Display Configuration

```
SAVE_INTERVAL = 100
```

Purpose: Save checkpoint every N cards

Impact:

- More frequent = Better recovery granularity but more disk I/O
- Less frequent = Less overhead but larger work loss on crash

Why 100? Good compromise

```
DISPLAY_INTERVAL = 100
```

Purpose: Update console progress every N cards

Why same as SAVE_INTERVAL? Coordinated checkpointing and user feedback

6. INPUT SPECIFICATION

Input File

Filename: cardmarket_card_list.json

Location: Same directory as script

Format: JSON array of card objects

Required Fields Per Card

```
{
  "expansion_id": 1433,
  "expansion_name": "2013 Zexal Collection Tin",
  "expansion_code": "ZTIN-EN",
  "card_id": 260915,
  "card_name": "Galaxy Wizard",
  "card_number": "002",
  "card_rarity": "Super Rare",
  "card_url": "https://www.cardmarket.com/en/YuGiOh/Products/Singles/..."
}
```

Field Validation Rules

expansion_id (integer)

- Must exist
- Identifies the card set

expansion_name (string)

- Must exist
- Human-readable set name

expansion_code (string)

- Must exist
- **Must not be empty string** (validated)
- Used to construct `card_set_number`

card_id (integer)

- Must exist
- **Must be unique** (duplicate check performed)
- Primary identifier

card_name (string)

- Must exist
- Full card name including variant info

card_number (string)

- Must exist
- **Must not be empty string** (validated)
- Card number within set (e.g., "002", "EN001")

card_rarity (string)

- Must exist

- Card rarity (e.g., "Super Rare", "Secret Rare")

card_url (string)

- Must exist
- **Must start with "https://"** (validated)
- Full URL to card's Cardmarket page

Input Validation

Duplicate Detection:

```
card_ids = [c.get('card_id') for c in cards]
duplicate_ids = [cid for cid in set(card_ids) if card_ids.count(cid) > 1]
if duplicate_ids:
    print("ERROR: Duplicates found!")
    exit()
```

Why critical? Duplicates would cause:

- Wasted processing (same card twice)
- Incorrect statistics
- Checkpoint confusion

Field Validation:

```
def validate_input_card(card):
    required = ['expansion_id', 'expansion_name', 'expansion_code',
               'card_id', 'card_name', 'card_number', 'card_rarity', 'card_url']

    for field in required:
        if field not in card:
            return False, f"Missing: {field}"

    if not str(card['expansion_code']).strip():
        return False, "Empty expansion_code"

    if not str(card['card_number']).strip():
        return False, "Empty card_number"

    if not card['card_url'].startswith('https://'):
        return False, "Invalid URL"

    return True, None
```

Validation failures result in rejection with reason "Failed - missing input data"

7. OUTPUT SPECIFICATION

Success Output File

Filename: cardmarket_card_details.json

Format: JSON array of successful card extractions

Success Output Structure

```
{
  "card_data": {
    "card_id": 260915,
    "card_name": "Galaxy Wizard",
    "card_rarity": "Super Rare",
    "card_number": "002",
    "card_set_number": "ZTIN-EN-EN002"
  },
  "expansion_data": {
    "expansion_id": 1433,
    "expansion_name": "2013 Zexal Collection Tin",
    "expansion_code": "ZTIN-EN"
  },
  "price_data": {
    "url": "https://www.cardmarket.com/en/YuGiOh/Products/Singles/...",
    "low_price": 0.50,
    "trend_price": 0.75,
    "avg_30_price": 0.70,
    "avg_7_price": 0.72,
    "avg_1_price": 0.74,
    "price_date": "2025-10-27",
    "currency": "EUR"
  }
}
```

Field Descriptions

card_set_number (string)

- Computed field: f"{expansion_code}-EN{card_number}"
- Example: "ZTIN-EN-EN002"
- Unique identifier combining set and card number

Prices (float)

- All prices stored as decimal numbers
- No currency symbols
- Decimal separator: dot (.)
- Examples: 0.50, 1.25, 100.00

price_date (string)

- Format: "YYYY-MM-DD"
- Extraction date (not market date)

currency (string)

- Always "EUR" (Cardmarket uses Euro)
- Stored for future multi-currency support

Rejection Output File

Filename: cardmarket_card_details_rejection.json

Format: JSON array of failed extractions with detailed reasons

Rejection Output Structure

```
{
  "card_id": 667379,
  "card_name": "Cubic Wave (V.2 - Secret Rare)",
  "card_url": "https://www.cardmarket.com/en/YuGiOh/...",
  "expansion_id": 4592,
  "expansion_name": "20th Anniversary Legend Collection",
  "expansion_code": "20TH-C",
  "rejection_reason": "Failed - N/A",
  "error_detail": "One or more price fields are N/A",
  "timestamp": "2025-10-27T20:51:12.232075",
  "attempts": 3
}
```

Rejection Categories

1. Failed - site unreachable

```
{
  "rejection_reason": "Failed - site unreachable",
  "error_detail": "HTTP 403 (attempt 3/3)"
}
```

Causes:

- HTTP 403 (Forbidden - Cloudflare block)
- HTTP 404 (Not Found)
- HTTP 429 (Too Many Requests)
- HTTP 500/503 (Server errors)
- Network timeouts
- Connection errors

Recoverable: Yes (attempted in Phase 2)

2. Failed - N/A

```
{
  "rejection_reason": "Failed - N/A",
  "error_detail": "One or more price fields are N/A"
}
```

Causes:

- One or more of the 5 price fields contain "N/A"
- Indicates no market data available for that price point
- Often occurs with new/rare cards with limited trading history

Recoverable: No (permanent condition - no market data exists)

3. Failed - missing data

```
{
  "rejection_reason": "Failed - missing data",
  "error_detail": "Price fields missing or invalid"
}
```

Causes:

- Less than 5 price fields found in HTML
- HTML structure different than expected
- Prices unparseable (corrupted data)

Recoverable: No (structural/data issue)

4. Failed - missing input data

```
{
  "rejection_reason": "Failed - missing input data",
  "error_detail": "Empty expansion_code"
}
```

Causes:

- Input validation failure
- Required field missing
- Empty expansion_code
- Empty card_number
- Invalid URL format

Recoverable: No (input data quality issue)

Checkpoint File

Filename: cardmarket_card_details_checkpoint.json

Purpose: Resume point tracking

Deleted: After Phase 1 completes successfully

```
{
  "last_processed_index": 5432,
  "phase1_complete": false
}
```

last_processed_index: Card array index of last saved card

phase1_complete: Boolean flag indicating Phase 1 completion status

8. PRICE EXTRACTION LOGIC

HTML Page Structure

Cardmarket card pages have a consistent structure for displaying prices:

```
<dl class="labeled">
  <dt class="col-6 col-xl-5">From</dt>
  <dd class="col-6 col-xl-7">0,50 €</dd>

  <dt class="col-6 col-xl-5">Price Trend</dt>
  <dd class="col-6 col-xl-7">0,75 €</dd>

  <dt class="col-6 col-xl-5">30-days average price</dt>
  <dd class="col-6 col-xl-7">0,70 €</dd>

  <dt class="col-6 col-xl-5">7-days average price</dt>
  <dd class="col-6 col-xl-7">0,72 €</dd>

  <dt class="col-6 col-xl-5">1-day average price</dt>
  <dd class="col-6 col-xl-7">0,74 €</dd>
</dl>
```

Label Matching Strategy

The script uses **substring matching** (case-insensitive) to find price fields:

```
label_map = {
  'from': 'low_price',          # Matches "From"
  'price trend': 'trend_price', # Matches "Price Trend"
  '30-day': 'avg_30_price',     # Matches "30-days average price"
  '7-day': 'avg_7_price',       # Matches "7-days average price"
  '1-day': 'avg_1_price'        # Matches "1-day average price"
}
```

Why substring? Label text may vary slightly (e.g., "30-day" vs "30 days"). Substring matching is more robust.

Parsing Algorithm

```
for dt in soup.find_all('dt'):
    dt_text = dt.get_text(strip=True).lower()

    for label, price_key in label_map.items():
        if label in dt_text: # Substring match
            dd = dt.find_next_sibling('dd') # Get corresponding value
            if dd:
                price_text = dd.get_text(strip=True)
                price_value, is_na = parse_price(price_text)

                found_count += 1

                if is_na:
                    na_count += 1
                elif price_value is not None:
                    prices[price_key] = price_value
                    valid_count += 1
            break # Stop after first match
```

Key points:

- Uses `find_next_sibling('dd')` to get value after label
- Increments `found_count` for any field found (even if N/A or invalid)
- Tracks separate counts for N/A and valid prices
- Stops after first label match (prevents duplicate matching)

Edge Cases Handled

Case 1: Price field missing entirely

```
<dt>From</dt>
<dd>0,50 €</dd>
<!-- Price Trend missing -->
<dt>30-days average price</dt>
<dd>0,70 €</dd>
```

Result: `found_count = 3` → Returns `(None, False)` → "Failed - missing data"

Case 2: Price field present but N/A

```
<dt>From</dt>
<dd>0,50 €</dd>
<dt>Price Trend</dt>
<dd>N/A</dd>
```

Result: `na_count = 1` → Returns `(None, True)` → "Failed - N/A"

Case 3: Price field present but unparseable

```
<dt>From</dt>
<dd>??</dd>
```

Result: `parse_price("??")` → `(None, False)` → `valid_count` not incremented → Returns `(None, False)` → "Failed - missing data"

Case 4: All prices valid

```
<!-- All 5 fields present with valid prices -->
```

Result: `valid_count = 5` → Returns `(prices_dict, False)` → Success

9. N/A DETECTION & PRIORITY SYSTEM

Why Priority Matters

Consider this scenario:

Card HTML:

- From: 2.50€ (valid)
- Price Trend: **Missing entirely**
- 30-day: N/A
- 7-day: N/A
- 1-day: N/A

Question: Should this be classified as:

- "Failed - missing data" (because Trend field missing)?
- "Failed - N/A" (because 3 fields are N/A)?

Answer: "Failed - N/A" (N/A takes priority)

Reason: The presence of N/A values indicates the card has no market data. The missing "Price Trend" is likely also because no data exists, not because of a page structure issue.

Priority Checking Order

```
# PRIORITY 1: Check for ANY N/A (highest priority)
if na_count > 0:
    return (None, True) # Has N/A

# PRIORITY 2: Check if all 5 fields found
```

```

if found_count < 5:
    return (None, False) # Missing fields

# PRIORITY 3: Check if all found prices valid
if valid_count < 5:
    return (None, False) # Invalid prices

# SUCCESS: All checks passed
return (prices_dict, False)

```

N/A String Variations

The script recognizes multiple N/A representations:

```
na_patterns = ['n/a', 'n.a.', 'na', '--', '---', '-', '-']
```

Why so many?

- Different localization versions
- Various dash types (hyphen, en-dash, em-dash)
- Case variations (handled by `.lower()`)

Examples in HTML:

```

<dd>N/A</dd>
<dd>n/a</dd>
<dd>N.A.</dd>
<dd>--</dd>
<dd>---</dd>
<dd>-</dd>
<dd>-</dd>

```

All recognized as N/A.

Mixed Valid/N/A Scenario

HTML:

```

<dt>From</dt>
<dd>2,50 €</dd>

<dt>Price Trend</dt>
<dd>3,00 €</dd>

<dt>30-days average price</dt>
<dd>N/A</dd>

<dt>7-days average price</dt>
<dd>2,75 €</dd>

```

```
<dt>1-day average price</dt>
<dd>2,80 €</dd>
```

Extraction counts:

- found_count = 5 (all fields present)
- na_count = 1 (30-day is N/A)
- valid_count = 4 (4 valid prices)

Result: Returns (None, True) → "Failed - N/A"

Why reject? Even ONE N/A means incomplete data. All 5 prices required for success.

Why Not Partial Data?

Decision: Reject cards with ANY N/A, don't save partial data

Reasons:

1. **Data consistency:** All cards should have complete 5-price dataset
2. **Analysis reliability:** Missing prices could skew statistical analysis
3. **User expectation:** Output implies complete data available
4. **Retry possibility:** N/A cards can be rechecked later when data appears

Alternative approach considered: Save with null values for N/A fields

Rejected because: Creates inconsistent data structure, complicates downstream usage

10. SESSION MANAGEMENT & RATE LIMITING

CloudScraper Integration

Purpose: Bypass Cloudflare anti-bot protection

How CloudScraper works:

1. Makes initial request to target site
2. Detects Cloudflare challenge
3. Executes JavaScript challenge
4. Obtains clearance cookie
5. Subsequent requests use cookie

Session creation:

```
scraper = cloudscraper.create_scraper(
    browser={'browser': 'chrome', 'platform': 'windows', 'desktop': True},
    delay=10
)
```

Parameters:

- `browser='chrome'`: Mimics Chrome browser fingerprint
- `platform='windows'`: Mimics Windows OS
- `desktop=True`: Desktop browser (not mobile)
- `delay=10`: Wait 10 seconds for Cloudflare challenge resolution

Session Headers

Standard headers set for each session:

```
scraper.headers.update({
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)...',
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.9',
    'Accept-Encoding': 'gzip, deflate, br',
    'DNT': '1',
    'Connection': 'keep-alive',
    'Upgrade-Insecure-Requests': '1',
    'Cache-Control': 'max-age=0'
})
```

Purpose of each header:

User-Agent: Identifies browser type/version

- 10 different User-Agents rotated across workers
- Simulates traffic from different browsers
- Reduces bot detection likelihood

Accept: Specifies accepted content types

- `text/html`: Primary content type
- `application/xhtml+xml`: XHTML format
- `application/xml`: XML format
- `*/*;q=0.8`: Anything else (lower priority)

Accept-Language: Preferred language

- `en-US,en;q=0.9`: English (US) preferred, English acceptable

Accept-Encoding: Accepted compression

- `gzip, deflate, br`: Brotli compression supported
- Reduces bandwidth usage

DNT: Do Not Track

- `1`: Request not to be tracked

- Privacy preference signal

Connection: Connection handling

- `keep-alive`: Reuse TCP connection (important for session pooling)

Upgrade-Insecure-Requests: HTTPS preference

- 1: Prefer HTTPS over HTTP

Cache-Control: Caching behavior

- `max-age=0`: Always fetch fresh content

Rate Limiting Implementation

Global coordinator pattern:

```
class RateLimiter:
    def __init__(self, requests_per_second):
        self.min_interval = 1.0 / requests_per_second  # 0.125s for 8 req/sec
        self.last_request = 0
        self.lock = threading.Lock()

    def wait(self, jitter=0.1):
        with self.lock:  # Exclusive access
            current_time = time.time()
            time_since_last = current_time - self.last_request

            # Enforce minimum interval
            if time_since_last < self.min_interval:
                sleep_time = self.min_interval - time_since_last
                time.sleep(sleep_time)

            # Add random jitter
            if jitter > 0:
                time.sleep(random.uniform(0, jitter))

            # Update timestamp
            self.last_request = time.time()
```

Thread coordination:

With 20 workers and 8 req/sec limit:

```
Worker 1 makes request at t=0.000s
  ↓ Lock acquired, interval=0.125s
Worker 2 tries to make request at t=0.050s
  ↓ Lock blocks, waits for Worker 1
Worker 1 releases lock at t=0.125s + jitter
  ↓ last_request updated
Worker 2 acquires lock at t=0.125s
  ↓ Checks: time_since_last = 0.125s >= 0.125s ✓
```

- ↓ No sleep needed
- ↓ Adds jitter, releases lock

Jitter purpose:

Without jitter:

```
t=0.000s: Worker 1 → Request
t=0.125s: Worker 2 → Request
t=0.250s: Worker 3 → Request
(Perfectly synchronized - suspicious pattern)
```

With jitter (0-0.1s):

```
t=0.000s: Worker 1 → Request
t=0.180s: Worker 2 → Request (0.125 + 0.055 jitter)
t=0.312s: Worker 3 → Request (0.250 + 0.062 jitter)
(Randomized timing - appears more human-like)
```

HTTP 403 Recovery Strategy

403 Detection and Response:

```
if response.status_code == 403:
    session_pool.mark_403(worker_id)
    session_pool.refresh_session(worker_id)

    if attempt < retries - 1:
        time.sleep(random.uniform(20, 30)) # Long cooldown
        scraper, _ = session_pool.get_session(worker_id) # Fresh session
        continue # Retry with new session
```

Why 403 happens:

- Cloudflare detects suspicious pattern
- Too many requests from same session
- Bot fingerprint detected
- IP temporarily blocked

Recovery mechanism:

1. Mark timestamp of 403 for this worker
2. Immediately discard and refresh session
3. Wait 20-30 seconds (Cloudflare cooldown period)
4. Create new session with different User-Agent
5. Retry request with fresh identity

Cooldown enforcement:

```
if worker_id in last_403_time:
    if time.time() - last_403_time[worker_id] < 30:
        # Less than 30 seconds since last 403
        # Force session recreation
```

Prevents reusing "hot" sessions that recently got blocked.

11. CHUNK-BASED PROCESSING ARCHITECTURE

The Iterator Invalidation Problem

Python's `as_completed()` behavior:

```
futures = {future1, future2, future3}
for f in as_completed(futures):
    # Process future

    # ✗ BUG: Adding during iteration
    futures.add(future4) # future4 will NEVER be yielded!
```

Why? `as_completed()` creates internal snapshot of futures at initialization. Modifications to input collection are ignored.

Attempted incremental submission (v1.4.4 bug):

```
# Submit initial 60 futures
for i in range(60):
    future = executor.submit(...)
    futures[future] = card

# Process and add new futures
for future in as_completed(futures):
    result = future.result()

    # ✗ Add new future
    new_future = executor.submit(...)
    futures[new_future] = next_card # NEVER PROCESSED!
```

Result: Only initial 60 futures processed, remaining 64,032 ignored. Script exits prematurely.

Chunk-Based Solution

Architecture:

```
CHUNK_SIZE = 300

for chunk_start in range(0, total_cards, CHUNK_SIZE):
```

```

chunk_end = min(chunk_start + CHUNK_SIZE, total_cards)
chunk_cards = cards[chunk_start:chunk_end]

# Submit ALL futures for this chunk
futures = {}
for card in chunk_cards:
    future = executor.submit(...)
    futures[future] = card

# Process ALL futures in this chunk
for future in as_completed(futures):
    result = future.result()
    # ... process ...

# Chunk complete - check for interrupt
# This is a safe stopping point

```

Key principles:

1. **Submit all before processing:** All chunk futures added to dict BEFORE `as_completed()` iteration begins
2. **Complete processing per chunk:** Entire chunk processed to completion
3. **Interrupt between chunks:** `KeyboardInterrupt` caught naturally between chunks
4. **Safe stopping points:** Chunk boundaries are clean exit points

Shutdown Time Calculation

Formula:

$$\text{Shutdown Time} = (\text{CHUNK_SIZE} / \text{Processing Speed}) \times \text{Active Workers Multiplier}$$

With current settings:

- Chunk size: 300 cards
- Processing speed: ~18,000 cards/hour = 5 cards/second
- Worst case (interrupt at chunk start): $300 / 5 = 60$ seconds
- Average case (interrupt mid-chunk): ~30 seconds
- Best case (interrupt at chunk end): ~3 seconds

Why 300 is optimal:

Chunk Size	Shutdown Time	Overhead Impact
50	10s	High (many chunks)
100	20s	Moderate
200	40s	Low
300	60s	Very Low

Chunk Size	Shutdown Time	Overhead Impact
500	100s	Minimal
1000	200s	Minimal (but too slow)

Trade-off: 300 provides acceptable shutdown time while maintaining high performance.

Interrupt Handling Flow

```
try:
    for chunk in chunks:
        # Process chunk
        with ThreadPoolExecutor() as executor:
            # Submit all futures
            # Process with as_completed()

            # Chunk complete, save checkpoint
            save_data(...)

            # Natural checkpoint - interrupt caught here if pressed

except KeyboardInterrupt:
    interrupted = True
    print("Finishing current chunk...")
```

What happens:

1. User presses Ctrl+C during chunk processing
2. Python raises KeyboardInterrupt
3. **ThreadPoolExecutor context manager waits for all submitted futures** (important!)
4. When all chunk futures complete, `__exit__()` finishes
5. Exception caught, interrupted flag set
6. Final save occurs
7. Graceful exit message displayed

User experience:

```
[ 245/64092] Success: 88 | Failed: 12 | 0.4% | 18000/h
^C
△ Interrupt received after completing chunk #2
  Finishing current chunk... (may take up to 60 seconds)

[300 futures finish processing over next 45 seconds...]

=====
GRACEFULLY STOPPED
=====
✓ Completed 2 chunks (600 cards processed)
✓ Progress saved to card index: 599
```

```
✓ Total processed: 300 cards
✓ Successful: 280
✓ Rejected: 20
Duration: 0:01:12
```

```
□ To resume: Simply run the script again
```

```
=====
```

Chunk Progress Tracking

Per-chunk variables:

```
chunk_number = 0

for chunk_start in range(0, total_cards, CHUNK_SIZE):
    chunk_number += 1
    chunk_end = min(chunk_start + CHUNK_SIZE, total_cards)
    chunk_cards = cards[chunk_start:chunk_end]

    # Process chunk

    # After chunk completion
    checkpoint = {
        'last_processed_index': last_saved_idx,
        'phase1_complete': False
    }
    save_data(successful_cards, rejections, checkpoint)
```

Checkpoint saved after every chunk (in addition to every 100 cards within chunk)

Benefits:

- Resume from exact chunk boundary
- Clear progress indication
- Minimal work duplication on resume

12. GRACEFUL SHUTDOWN MECHANISM

User Interrupt Flow

Step 1: User presses Ctrl+C

```
User: [Ctrl+C pressed]
↓
Python: Raises KeyboardInterrupt exception
↓
Script: Exception propagates up call stack
```

Step 2: Exception caught in phase1_fast_run()

```

try:
    for chunk in process_chunks():
        # Process chunk with ThreadPoolExecutor

except KeyboardInterrupt:
    interrupted = True
    print("Interrupt received after completing chunk #X")
    print("Finishing current chunk... (may take up to 60 seconds)")

```

Step 3: ThreadPoolExecutor cleanup

```

with ThreadPoolExecutor(max_workers=20) as executor:
    # Futures being processed...

# When context manager exits:
# 1. No new futures accepted
# 2. Wait for all submitted futures to complete
# 3. Collect all results
# 4. Close executor

```

Important: Python's `ThreadPoolExecutor.__exit__()` waits for all active futures. Cannot be bypassed (by design for thread safety).

Step 4: Save final state

```

checkpoint = {
    'last_processed_index': last_saved_idx,
    'phase1_complete': False # Interrupted, not complete
}
save_data(successful_cards, rejections, checkpoint)

```

Step 5: Display summary

```

print(" GRACEFULLY STOPPED")
print(f"Completed {chunk_number} chunks")
print(f"Progress saved to card index: {last_saved_idx}")
print("To resume: Simply run the script again")

```

Step 6: Return None signal

```

return None # Special return value indicating interruption

```

Step 7: main() checks return value

```

result = phase1_fast_run(cards, start_idx)

if result is None: # Interruption detected

```

```
print("Script interrupted gracefully. All progress saved.")
return  # Exit main()
```

Why Shutdown Takes Time

Cannot instantaneously stop because:

1. Active HTTP requests in flight

- 20 workers may have requests in progress
- Each request has 15-second timeout
- Must wait for completion (can't forcibly terminate)

2. Thread safety requirements

- Cannot kill threads (unsafe, causes resource leaks)
- Must let threads finish their current task
- Ensures data consistency

3. Result collection

- ThreadPoolExecutor collects results from all futures
- Ensures no data loss
- Takes time proportional to remaining futures

4. File I/O operations

- Saving large JSON files (potentially thousands of records)
- Disk write operations not instant
- Must complete to avoid corruption

Theoretical minimum shutdown time:

```
Min = max(active_request_timeouts) + file_save_time + overhead
    ≈ 15s (timeout) + 1-3s (save) + 1-2s (cleanup)
    ≈ 17-20 seconds
```

Typical shutdown time: 30-60 seconds (depends on when interrupt occurs in chunk processing)

Second Ctrl+C (Force Quit)

Behavior:

```
if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
```

```
# This catches SECOND Ctrl+C
print("△ Script terminated during startup/loading")
```

First Ctrl+C: Caught in `phase1_fast_run()`, graceful shutdown initiated

Second Ctrl+C: Caught in `main()`, immediate termination

Warning: Second Ctrl+C may cause data loss!

- Pending file writes interrupted
- Checkpoint may be incomplete
- Last 100 cards (since last save) lost

Recommendation: Wait for first Ctrl+C to complete (max 60s). Only use second Ctrl+C in emergency.

Interrupt During Phase 2

Phase 2 is sequential (1 worker), simpler interrupt handling:

```
try:
    for idx, rejection in enumerate(recoverable):
        # Process card (may take up to 4 minutes per card)
        result = process_card_worker(...)

        # Save every 10 cards
        if (idx + 1) % 10 == 0:
            save_data(...)

except KeyboardInterrupt:
    print("Phase 2 interrupted - saving progress...")

    # Add unprocessed cards back to rejection list
    still_rejected.extend(recoverable[idx+1:])

    # Save final state
    save_data(...)
```

Phase 2 shutdown time:

- Worst case: Up to 4 minutes (if card has slow retries in progress)
- Typical: 30-60 seconds
- Saved after every 10 cards, minimal data loss

13. ERROR HANDLING & RETRY LOGIC

HTTP Status Code Handling

200 OK:

```
if response.status_code == 200:
    return response.text, None # Success
```

Normal successful response.

403 Forbidden:

```
elif response.status_code == 403:
    session_pool.mark_403(worker_id)
    session_pool.refresh_session(worker_id)

    if attempt < retries - 1:
        time.sleep(random.uniform(20, 30)) # Long cooldown
        scraper, _ = session_pool.get_session(worker_id)
        continue # Retry with fresh session

    return None, "HTTP 403 (attempt 3/3)"
```

Causes:

- Cloudflare detected bot behavior
- Session flagged as suspicious
- Rate limit exceeded for this session

Recovery:

- Force session refresh (new identity)
- Wait 20-30 seconds (Cloudflare cooldown)
- Retry with completely fresh session

404 Not Found:

```
elif response.status_code == 404:
    return None, "HTTP 404" # Immediate failure, no retry
```

Causes:

- URL incorrect/outdated
- Card page removed from Cardmarket
- Input data issue

No retry: 404 is permanent (page doesn't exist)

429 Too Many Requests:

```
elif response.status_code == 429:
    time.sleep(10 * (attempt + 1)) # Escalating backoff
    continue # Retry
```

Causes:

- Global rate limit exceeded
- Too many requests from IP

Recovery:

- Wait 10 seconds on attempt 1
- Wait 20 seconds on attempt 2
- Wait 30 seconds on attempt 3
- Retry after cooldown

500/503 Server Errors:

```
elif response.status_code in [500, 503]:
    time.sleep(5) # Brief wait
    continue # Retry
```

Causes:

- Temporary server issue
- Database problem
- Server overload

Recovery:

- Short wait (5 seconds)
- Retry immediately
- Usually transient issues

Other Status Codes:

```
else:
    return None, f"HTTP {response.status_code}"
```

Treated as failure, returned for categorization.

Network Exception Handling

```
except Exception as e:
    error_detail = f"{type(e).__name__}"

    if attempt < retries - 1:
```

```
        time.sleep(2) # Short recovery wait
        continue # Retry

    return None, error_detail # Final attempt failed
```

Common exceptions:

requests.Timeout:

- Cause: Server didn't respond within 15 seconds
- Recovery: Retry (may be transient network issue)

requests.ConnectionError:

- Cause: Network connectivity problem
- Recovery: Retry (network may recover)

cloudscraper.exceptions.CloudflareError:

- Cause: Cloudflare challenge failed
- Recovery: Retry with fresh session

Other exceptions:

- Generic handling
- Retry for all exception types
- Logged in error_detail

Retry Decision Tree

```
Fetch card page
↓
HTTP Status = ?
├─ 200: Return HTML ✓
├─ 403: Refresh session, wait 20-30s, retry
├─ 404: Return error immediately (no retry)
├─ 429: Wait 10s × attempt, retry
├─ 500/503: Wait 5s, retry
├─ Exception: Wait 2s, retry
└─ Other: Return error

If all retries exhausted:
↓
Phase 1: Mark as "Failed - site unreachable"
↓
Phase 2: Attempt recovery with careful approach
├─ Max 5 retries (vs 3 in Phase 1)
├─ Escalating delays: 15, 30, 45, 60, 90 seconds
└─ Sequential processing (less aggressive)

If Phase 2 also fails:
```

↓
Final rejection: "Failed - site unreachable"

Validation Error Handling

Input validation failures:

```
is_valid, validation_error = validate_input_card(card)

if not is_valid:
    return {
        'success': False,
        'rejection_reason': 'Failed - missing input data',
        'error_detail': validation_error # e.g., "Empty expansion_code"
    }
```

No retry: Validation errors are permanent (input data issue)

Examples:

- "Missing: card_url"
- "Empty expansion_code"
- "Invalid URL"

Extraction Error Handling

Price extraction failures:

```
prices, has_na = extract_price_data(html)

if has_na:
    return {
        'success': False,
        'rejection_reason': 'Failed - N/A',
        'error_detail': 'One or more price fields are N/A'
    }

if prices is None:
    return {
        'success': False,
        'rejection_reason': 'Failed - missing data',
        'error_detail': 'Price fields missing or invalid'
    }
```

No retry: Extraction errors are permanent (data/structure issue)

Distinction:

- **N/A:** Data exists but marked as N/A (no market data)
- **Missing data:** Data structure unexpected or corrupted

14. PHASE 1: FAST PARALLEL EXTRACTION

Objectives

1. **Speed:** Process cards as quickly as possible
2. **Efficiency:** Minimize overhead and wasted resources
3. **Robustness:** Handle transient errors with retries
4. **Graceful degradation:** Accept some failures for speed (Phase 2 recovers)

Execution Flow

```
Phase 1 Start
↓
Load existing data (successful_cards, rejections, processed_ids)
↓
Filter cards_to_process (exclude already processed by card_id)
↓
Split into chunks of 300 cards
↓
For each chunk:
  └─ Create ThreadPoolExecutor (20 workers)
  └─ Submit all 300 futures
  └─ Process with as_completed():
    └─ Validate input
    └─ Fetch HTML (max 3 retries, rate limited)
    └─ Extract prices (N/A priority check)
    └─ Categorize result (success/rejection)
    └─ Display progress every 100 cards
    └─ Save checkpoint every 100 cards
  └─ Save chunk checkpoint
  └─ Check for KeyboardInterrupt
↓
If interrupted: Save final state, return None
↓
If complete: Return (successful_cards, rejections, stats)
```

Worker Thread Behavior

Each worker:

```
Worker N starts
↓
Assigned card from chunk
↓
Get session from pool (or create new)
↓
Validate card input
  └─ Valid: Continue
  └─ Invalid: Return rejection immediately
```

```

↓
Attempt 1:
├─ Wait for rate limiter
├─ Fetch card HTML (timeout 15s)
├─ Handle status codes/exceptions
├─ Extract prices
└─ Return result
↓
If failed and retries remain:
├─ Wait 3-5 seconds
├─ Attempt 2 (same process)
├─ Wait 3-5 seconds
└─ Attempt 3 (final)
↓
Return best result (success or final rejection)

```

Concurrency Pattern

20 workers operating simultaneously:

```

Timeline:

t=0.000s: Worker 1 starts card 1
t=0.125s: Worker 2 starts card 2    (rate limited: 1/8 second apart)
t=0.250s: Worker 3 starts card 3
...
t=2.375s: Worker 20 starts card 20
t=2.500s: Worker 1 finishes card 1, starts card 21
t=2.625s: Worker 2 finishes card 2, starts card 22
...

(Pattern continues until chunk exhausted)

```

Actual behavior:

- Not perfectly synchronized (request durations vary)
- Some workers finish faster (successful cards ~5s)
- Some slower (retries, 403 delays ~30s+)
- Rate limiter coordinates to maintain 8 req/sec global limit

Progress Display

Display every 100 cards:

```

[ 100/64092] Success: 88 | Failed: 10 | N/A: 2      | 0.2% | 18000/h | ETA: 3h 20m
[ 200/64092] Success: 95 | Failed: 5              | 0.3% | 18500/h | ETA: 3h 15m
[ 300/64092] Success: 100                        | 0.5% | 19000/h | ETA: 3h 10m

```

Batch counters reset after each display:

```

batch_success = 0
batch_failed = 0
batch_na = 0

for result in as_completed(futures):
    if result['status'] == 'success':
        batch_success += 1
    elif 'N/A' in result['rejection_reason']:
        batch_na += 1
    else:
        batch_failed += 1

    total_completed += 1

    if total_completed % 100 == 0:
        # Build status string (hide zeros)
        status_parts = []
        if batch_success > 0:
            status_parts.append(f"Success: {batch_success}")
        if batch_failed > 0:
            status_parts.append(f"Failed: {batch_failed}")
        if batch_na > 0:
            status_parts.append(f"N/A: {batch_na}")

        print(f"[{total_completed:5}/{total_cards:5}] {status_text:40} ...")

        # Reset counters
        batch_success = 0
        batch_failed = 0
        batch_na = 0

```

If all 100 succeeded:

```
[ 100/64092] Success: 100 | 0.2% | 18000/h
```

Zero values hidden for cleaner output.

Checkpoint Strategy

Two checkpoint types:

1. Periodic checkpoint (every 100 cards):

```

if total_completed % 100 == 0:
    checkpoint = {
        'last_processed_index': current_index,
        'phase1_complete': False
    }
    save_data(successful_cards, rejections, checkpoint)

```

2. Chunk completion checkpoint:

```
# After each chunk fully processed
checkpoint = {
    'last_processed_index': last_saved_idx,
    'phase1_complete': False
}
save_data(successful_cards, rejections, checkpoint)
```

Why both?

- Periodic: Minimizes work loss during chunk processing
- Chunk: Ensures clean resume point at